

# **Software Implementation and Testing Document**

**For**

**Group 1**

Version 2.0

**Authors:**

Jaliah Meade  
Nikki Zahedi  
Isabella Maldur  
Annika Needham

### **1. Programming Languages (5 points)**

*List the programming languages use in your project, where you use them (what components of your project) and your reason for choosing them (whatever that may be).*

Python handled backend logic such as routing, session management, authentication, and processing requests while interacting with the database. HTML was used to structure web pages, including forms and layouts for user interaction like the login page, and CSS was used to style and enhance the visual design. SQL was used to create and manage the database, including storing and organizing data for employees, shifts, availability, and swap requests.

### **2. Platforms, APIs, Databases, and other technologies used (5 points)**

*List all the platforms, APIs, Databases, and any other technologies you use in your project and where you use them (in what components of your project).*

Flask was used to develop the web application and handle page navigation between different components. SQLite was used as the database system to store and organize all application data.

### **3. Execution-based Functional Testing (10 points)**

*Describe how/if you performed functional testing for your project (i.e., tested for the **functional requirements** listed in your RD).*

Functional testing was performed by verifying that employees and admins could log in with valid credentials and that error messages appeared for invalid IDs or passwords. Password functionality was tested by checking that users were limited to attempts and redirected appropriately, while admins could log in and reset passwords without restrictions. Additional testing included confirming that the dashboard loaded correctly, navigation between pages worked, availability could be added, edited, and deleted, and shift swap requests could be submitted, approved, denied, and updated properly.

### **4. Execution-based Non-Functional Testing (10 points)**

*Describe how/if you performed non-functional testing for your project (i.e., tested for the **non-functional requirements** listed in your RD).*

Non-functional testing was performed by checking that pages such as the login, dashboard, and availability loaded quickly and responded smoothly to user actions. Input validation was tested to ensure the system handled incorrect or missing data without crashing. The system was also tested with multiple users interacting with features like shift swaps and availability updates to confirm consistent performance and proper handling of simultaneous actions.

### **5. Non-Execution-based Testing (10 points)**

*Describe how/if you performed non-execution-based testing (such as code reviews/inspections/walkthroughs).*

Non-execution-based testing was performed through group discussions and walkthroughs of key features such as admin authentication, password handling, and shift swap functionality. The team reviewed each other's code to check for logical errors, consistency, and proper integration between components like the dashboard, availability management, and request handling.